SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 0605 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO1: *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks: 25**

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

*I **K.KANISHKA(192511259)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
K.KANISHKA

SET-2

1. A machine learning model shows the following loss values during training: (5 Marks)

(a) Epoch 1: Training Loss = 3.0, Validation Loss = 2.7
(b) Epoch 2: Training Loss = 2.6, Validation Loss = 2.3
(c) Epoch 3: Training Loss = 2.3, Validation Loss = 2.1
(d) Epoch 4: Training Loss = 2.1, Validation Loss = 2.0
(e) Epoch 5: Training Loss = 1.9, Validation Loss = 2.2
(f) Epoch 6: Training Loss = 1.7, Validation Loss = 2.5
(g) Epoch 7: Training Loss = 1.6, Validation Loss = 2.8
(h) Epoch 8: Training Loss = 1.4, Validation Loss = 3.1

i) At which epoch does the model achieve optimal generalization?
ii) Identify signs of overfitting from the data.
iii) Recommend two regularization techniques and explain their impact.

CODING:

```
# Training and Validation Loss Analysis

training_loss = [3.0, 2.6, 2.3, 2.1, 1.9, 1.7, 1.6, 1.4]
validation_loss = [2.7, 2.3, 2.1, 2.0, 2.2, 2.5, 2.8, 3.1]

print("Epoch\tTraining Loss\tValidation Loss")

for i in range(len(training_loss)):
    print(i+1, "\t", training_loss[i], "\t\t", validation_loss[i])

# Find minimum validation loss
best_epoch = validation_loss.index(min(validation_loss)) + 1

print("\nOptimal Generalization at Epoch:", best_epoch)

# Check Overfitting
for i in range(1, len(validation_loss)):
    if validation_loss[i] > validation_loss[i-1]:
        print("\nOverfitting starts from Epoch", i+1)
        break

print("\nRecommended Regularization Techniques:")
print("1. Early Stopping")
print("2. Dropout")
print("3. L2 Regularization")
```

OUTPUT:

```
====== RESTART: C:/Users/hp/OneDrive/Desktop/MACHINE LEARNING/ASS 1 (1).py =====
Epoch   Training Loss   Validation Loss
1        3.0             2.7
2        2.6             2.3
3        2.3             2.1
4        2.1             2.0
5        1.9             2.2
6        1.7             2.5
7        1.6             2.8
8        1.4             3.1

Optimal Generalization at Epoch: 4

Overfitting starts from Epoch 5

Recommended Regularization Techniques:
1. Early Stopping
2. Dropout
3. L2 Regularization
```

2. In an industrial process, an AI robot is assigned to segregate the front and back tires automatically. Let the number of tires are 250 (125 front and 125 back tires). (5 Marks)
a. Front identified as Front: 119
a. Front identifies as Back: 06
a. Back identified as Back: 120
a. Back identified as Front: 05
Compute: Accuracy, Precision, Sensitivity, Specificity and F1-Score.

CODING:

ASS 1 (2).py - C:/Users/hp/OneDrive/Desktop/MACHINE LEARNING/ASS 1 (2).py (3

File Edit Format Run Options Window Help

```
# Tire Classification Performance

TP = 119
FN = 6
TN = 120
FP = 5

accuracy = (TP + TN) / (TP + TN + FP + FN)

precision = TP / (TP + FP)

recall = TP / (TP + FN)

specificity = TN / (TN + FP)

f1 = 2 * precision * recall / (precision + recall)

print("Accuracy :", round(accuracy,4))
print("Precision :", round(precision,4))
print("Sensitivity :", round(recall,4))
print("Specificity :", round(specificity,4))
print("F1 Score :", round(f1,4))
```

OUTPUT:

```
====== RESTART: C:/Users/hp/OneDrive/Desktop/MACHINE LEARNING/ASS 1 (2).py =====
Accuracy : 0.956
Precision : 0.9597
Sensitivity : 0.952
Specificity : 0.96
F1 Score : 0.9558
```

3. In a medical diagnosis system, an AI model detects whether a patient has a disease or not. Out of 300 cases (150 diseased and 150 healthy): (5 Marks)
a. Diseased identified as Diseased: 138
a. Diseased identified as Healthy: 12
a. Healthy identified as Healthy: 135
a. Healthy identified as Diseased: 15
Compute: Accuracy, Precision, Sensitivity (Recall), Specificity, and F1-Score.

CODING:

ASS 1 (3).py - C:/Users/hp/OneDrive/Desktop/MACHINE LEARNING/ASS 1 (3).py (3.13.14)

File Edit Format Run Options Window Help

```
# Medical Diagnosis Performance

TP = 138
FN = 12
TN = 135
FP = 15

accuracy = (TP + TN) / (TP + TN + FP + FN)

precision = TP / (TP + FP)

recall = TP / (TP + FN)

specificity = TN / (TN + FP)

f1 = 2 * precision * recall / (precision + recall)

print("Accuracy :", round(accuracy,4))
print("Precision :", round(precision,4))
print("Sensitivity :", round(recall,4))
print("Specificity :", round(specificity,4))
print("F1 Score :", round(f1,4))
```

OUTPUT:

```
====== RESTART: C:/Users/hp/OneDrive/Desktop/MACHINE LEARNING/ASS 1 (3).py =====
Accuracy : 0.91
Precision : 0.902
Sensitivity : 0.92
Specificity : 0.9
F1 Score : 0.9109
```

4. In a spam email filtering system, an AI model classifies emails as Spam or Not Spam. Out of 400 emails (200 spam and 200 non-spam): (5 Marks)

- Spam identified as Spam: 180
- Spam identified as Not Spam: 20
- Not Spam identified as Not Spam: 170
- Not Spam identified as Spam: 30

Compute: Accuracy, Precision, Sensitivity, Specificity, and F1-Score.

CODING:

ASS 1 (4).py - C:/Users/hp/OneDrive/Desktop/MACHINE LEARNING/ASS 1 (4).py (3.13.14)

File Edit Format Run Options Window Help

```
# Spam Email Classification

TP = 180
FN = 20
TN = 170
FP = 30

accuracy = (TP + TN) / (TP + TN + FP + FN)

precision = TP / (TP + FP)

recall = TP / (TP + FN)

specificity = TN / (TN + FP)

f1 = 2 * precision * recall / (precision + recall)

print("Accuracy :", round(accuracy,4))
print("Precision :", round(precision,4))
print("Sensitivity :", round(recall,4))
print("Specificity :", round(specificity,4))
print("F1 Score :", round(f1,4))
```

OUTPUT:

```
>>>
====== RESTART: C:/Users/hp/OneDrive/Desktop/MACHINE LEARNING/ASS 1 (4).py =====
Accuracy : 0.875
Precision : 0.8571
Sensitivity : 0.9
Specificity : 0.85
F1 Score : 0.878
>>>
```

5. In a quality inspection system in manufacturing, an AI model identifies defective and non-defective products. Out of 500 products (250 defective and 250 non-defective): Marks)

- Defective identified as Defective: 2 30
- Defective identified as Non-Defective: 20
- Non-Defective identified as Non-Defective: 225
- Non-Defective identified as Defective: 25

Compute: Accuracy, Precision, Sensitivity, Specificity, and F1-Score.

CODING:

ASS 1 (5).py - C:/Users/hp/OneDrive/Desktop/MACHINE LEARNING/ASS 1 (5).py (3.13.14)

File Edit Format Run Options Window Help

```
# Manufacturing Quality Inspection

TP = 230
FN = 20
TN = 225
FP = 25

accuracy = (TP + TN) / (TP + TN + FP + FN)

precision = TP / (TP + FP)

recall = TP / (TP + FN)

specificity = TN / (TN + FP)

f1 = 2 * precision * recall / (precision + recall)

print("Accuracy :", round(accuracy,4))
print("Precision :", round(precision,4))
print("Sensitivity :", round(recall,4))
print("Specificity :", round(specificity,4))
print("F1 Score :", round(f1,4))
```

OUTPUT:

```
====== RESTART: C:/Users/hp/OneDrive/Desktop/MACHINE LEARNING/ASS 1 (5).py =====
Accuracy : 0.91
Precision : 0.902
Sensitivity : 0.92
Specificity : 0.9
F1 Score : 0.9109
```